

Practical no:3

Aim: Comparison of Classification Accuracy of SVM for given dataset

Theory:

Support Vector Machines (SVMs) are a set of supervised learning methods used for classification, regression, and outliers detection. The primary goal of the SVM algorithm is to find a hyperplane in an N-dimensional space (N is the number of features) that distinctly classifies the data points.

Key concepts:

- **Hyperplane:** In SVM, a hyperplane is a decision boundary that helps classify the data points. Data points falling on either side of the hyperplane can be attributed to different classes.
- **Support Vectors:** Data points that are closer to the hyperplane and influence the position and orientation of the hyperplane. These are the critical elements of the dataset.
- **Margin:** The distance between the hyperplane and the closest data points from either class. SVM aims to maximize this margin.
- **Import Libraries:** Import necessary libraries such as numpy, pandas, matplotlib, and sklearn.
- **Load the Dataset:** Load and explore the dataset to understand its structure and features.
- **Preprocess the Data:** Handle missing values, encode categorical variables, and split the data into training and testing sets.
- **Train the SVM Model:** Train the SVM model using different kernel functions (e.g., linear, polynomial, RBF).
- **Evaluate the Model:** Evaluate the model's performance using classification accuracy and other relevant metrics.
- **Compare Results:** Compare the classification accuracy of the SVM model with different kernel functions.

Program:

```
#Comparison of Classification Accuracy of SVM for given dataset using  
iris dataset
```

```
import numpy as np  
from sklearn import datasets  
from sklearn.model_selection import train_test_split  
from sklearn.svm import SVC  
from sklearn.metrics import accuracy_score
```

```
# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=62)

# Define SVM models with different kernels
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
accuracies = {}

for kernel in kernels:
    # Initialize and train the SVM model
    model = SVC(kernel=kernel, random_state=62)
    model.fit(X_train, y_train)

    # Make predictions
    y_pred = model.predict(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)
    accuracies[kernel] = accuracy
    print(f'Accuracy with {kernel} kernel: {accuracy:.14f}')

# Display results
print("\nComparison of Classification Accuracy:")
for kernel, accuracy in accuracies.items():
    print(f"Kernel: {kernel} - Accuracy: {accuracy}
```

output:

Accuracy with linear kernel: 0.9866
Accuracy with poly kernel: 0.9866
Accuracy with rbf kernel: 0.9733
Accuracy with sigmoid kernel: 0.2933

Comparison of Classification Accuracy:
Kernel: linear - Accuracy: 0.9866
Kernel: poly - Accuracy: 0.9866
Kernel: rbf - Accuracy: 0.9733
Kernel: sigmoid - Accuracy: 0.2933